

GaganMitra — 15-Day Prep Plan + 30-Hour Execution Plan

Team Mavericks (Mansi Pillai, Priyamvada Singh) — Supernova Grand Finale, IET DAVV Indore

0. What Was Fixed From the Rough Draft

Issue in draft	Fix applied here
Phase 0-8 (multi-day) and the 30-hour plan (Section 8) both describe the same work at different speeds, with no explicit link between them.	Split cleanly into two connected documents below: a 15-day PREP plan (Section 2) that ends with a rehearsed, working pipeline, and a 30-HOUR plan (Section 3) that assumes that pipeline exists and focuses on adapting it to the live scenario, polishing, and pitching.
Tasks are described as one combined stream, not split by person.	Every timeline below runs two explicit swimlanes (Mansi / Priyamvada) with named sync points, so you always know who is doing what at the same time.
Recommended dashboard stack (Streamlit + Plotly) is Python-first, but Priyamvada's existing strength is HTML.	Addressed head-on in Section 1 as an explicit early decision, not a default.
No mention of whether pre-building before the official 30-hour window is permitted.	Flagged explicitly in Section 1 as an assumption to confirm with organizers before committing 15 days of build time.

1. Two Decisions to Make on Day 1 (Before Anything Else)

1.1 Confirm the pre-build rule with organizers

Assumption used throughout this plan: the 15-day period is for learning, environment setup, data auditing, and building a rehearsed practice pipeline — not for writing the literal final submission code before the clock starts. Confirm this with Supernova organizers directly. If unrestricted pre-building is explicitly allowed, the 15 days can produce a near-final system and the 30-hour window becomes adaptation + polish + pitch. If it is NOT allowed, keep your prep-phase code in a clearly separate practice repo, and treat the 15 days purely as a rehearsal so that on hackathon day you are executing a known plan fast, not learning it live — this is still a legitimate and large advantage even if no prep code carries over directly.

1.2 Dashboard stack: play to Priyamvada's actual strength

The draft defaults to Streamlit + Plotly, which are Python tools — this doesn't use Priyamvada's HTML skill at all, and it sidelines one of your two team members into "assisting" rather than owning a track.

- **Recommended:** Priyamvada builds a custom HTML/CSS/JS dashboard using Chart.js or Plotly.js, which read a JSON alert-bundle file exported by Mansi's Python pipeline. This is a real skill match, gives more visual control for the demo (a custom-styled dashboard reads as more polished to judges than a default Streamlit theme), and cleanly decouples the two tracks: Mansi's pipeline just needs to produce a well-defined JSON schema; Priyamvada's dashboard just needs to consume it. Neither track blocks the other once the JSON schema is agreed (do this on Day 1).
- **Fallback safety net:** keep a minimal Streamlit skeleton (a few dozen lines) as backup in case the custom dashboard risks not being demo-ready in time. Building it takes under an hour and it directly reuses the same JSON schema, so it costs little to keep as insurance.

Note: This decision should not be revisited mid-prep — agree it on Day 1 and commit, per the draft's own "freeze scope" principle.

2. 15-Day Prep Plan (Swimlane by Person)

Each day lists what Mansi and Priyamvada do in parallel, plus the sync point where their work must agree (usually a schema or interface, not a finished feature).

Day	Mansi (Python / ML track)	Priyamvada (Dashboard / product track)	Sync point
1	Set up environment, install stack (pandas, PyOD, PyTorch, etc.). Download ESA-ADB, confirm it loads. Log licence/citation info.	Learn Chart.js or Plotly.js basics (pick one, per Section 1.2). Set up a static HTML/CSS/JS project skeleton with dummy data.	Agree the JSON alert-bundle schema together (fields: event ID, time window, severity, contributing channels, score, subsystem hypothesis) before either track writes real code.
2-3	Data audit: inspect timestamps, channels, missing values on ESA-ADB. Produce the data-quality report and 5 representative plots.	Build the static dashboard shell against the agreed JSON schema, using placeholder/fake alert data matching that schema exactly.	Priyamvada's placeholder JSON should be byte-for-byte the same shape Mansi's pipeline will eventually output — check this explicitly, not just "looks similar."
4-5	Build preprocessing (sorting, gap handling, missingness indicators, sliding windows) and the two baseline detectors (robust z-score/EWMA, ECOD or Isolation Forest).	Add telemetry-replay charts and an active-anomaly table to the dashboard, still on placeholder data. Start the synthetic-simulator UI concept (how faults will be shown).	Mansi shares one real (not placeholder) baseline output as JSON; Priyamvada swaps it in and fixes any schema mismatches immediately.
6-8	Build the LSTM/TCN autoencoder, train on normal windows, calibrate threshold, compare against baselines. If unstable, this is where the fallback rule applies (Section 3.1).	Build event-detail view, contributing-channel chart, and subsystem-hypothesis panel. Start styling pass (colors, layout, mission-control look) using the frontend-design sensibility.	Mid-point check: does the real model's output still fit the agreed schema? Adjust the schema together if not — better now than hour 20 of the hackathon.
9-10	Add persistence, hysteresis, event grouping, severity scoring, contributing-channel ranking, subsystem mapping.	Build the alert-bundle download feature (JSON/HTML export) and the metrics/evaluation display page.	Full real pipeline output (event objects, not just raw scores) flows into the dashboard for the first time.
11	Build the synthetic spacecraft simulator with labelled fault injection (per the fault list in the original draft's Data Strategy section).	Wire the dashboard to switch between ESA-ADB replay and synthetic-simulator replay.	Confirm both data sources produce schema-compatible output through the same dashboard.
12	Run the full chronological evaluation: precision/recall/F1/event-F1/false-alarms/detection delay/runtime/memory, exactly as specified in the draft's Phase 7.	Polish visual design pass: consistent typography, color tokens, responsive layout, loading states for a live demo.	None required — independent work day.
13	Investigate the Chandrayaan-3 APXS housekeeping data (Tier 2). Decide honestly whether it's feasible to ingest, or stays a stated future-work line.	Build the "mission overview" landing page and finalize navigation between all dashboard pages listed in the draft's Phase 6.	None required — independent work day.
14	FULL DRY RUN: simulate the 30-hour plan (Section 3) end-to-end at real speed, using real ESA-ADB data, timing each phase.	Same dry run, in parallel, on the dashboard side.	This IS the sync point — treat it as a rehearsal of the actual hackathon day, not a casual test. Note where you went over time and fix the plan, not just the code.
15	Fix whatever broke in the Day 14 dry run. Cache all demo outputs (data, model results, plots, alert bundles) locally so hackathon day never depends on network/compute luck.	Same: fix dashboard issues found in the dry run, finalize a working offline build.	Final joint check: can the whole demo run from a clean environment, offline, in under 60 seconds of setup? Rehearse the 2-minute pitch together.

Note: Colour key: green-tinted cells = Mansi's track, orange-tinted cells = Priyamvada's track.

3. 30-Hour Hackathon Execution Plan (Swimlane by Person)

This assumes Section 2 is complete: a working, rehearsed pipeline and dashboard already exist. The 30 hours are for adapting to the live scenario/dataset given on the day, integrating, polishing, and pitching — not building from zero.

3.1 Fallback rule (unchanged from the draft — it's correct)

A reliable statistical/unsupervised detector plus a polished alert bundle beats an unstable deep-learning model. The temporal model is optional; the operational workflow is mandatory. If the LSTM autoencoder is not stable by the checkpoint below, freeze the baseline and move on — do not keep debugging it into the integration window.

Hou rs	Mansi (Python / ML track)	Priyamvada (Dashboard / product track)	Sync point
0-2	Confirm the live scenario/dataset given on the day. Adapt data loading if the format differs from your rehearsed ESA-ADB path. Re-confirm the JSON schema still fits.	Confirm dashboard still loads the rehearsed placeholder data. Prepare to swap in live data the moment it's ready.	Agree the minimum success condition for this specific event/scenario.
2-6	Run data audit on the live scenario data; adapt preprocessing if needed.	Adjust any dashboard labels/branding for the specific scenario given. Keep the shell otherwise untouched.	None required.
6-10	Run baselines (z-score/EWMA, ECOD/Isolation Forest) on the live data; produce first real anomaly events.	Swap dashboard from placeholder to first real baseline output.	First real end-to-end pass: baseline events visible in the actual dashboard.
10-16	Attempt the temporal model (LSTM/TCN autoencoder). Apply the fallback rule at hour 16 without exception if it's unstable.	Continue polishing event-detail and contributing-channel views using real baseline data while the temporal model trains.	Checkpoint at hour 16: go/no-go on the temporal model.
16-20	Add persistence, severity, event grouping, contributing-channel ranking, subsystem mapping to the real pipeline.	Wire the alert-bundle download and metrics page to real output.	Second full pass: real events with severity/evidence visible end-to-end.
20-25	Support integration; fix pipeline-side bugs the dashboard surfaces.	Final dashboard integration: all pages, all real data, alert download working.	This is the main integration window — both people should be looking at the same screen for at least part of it.
25-28	Run the final chronological evaluation; produce the metrics table (precision, recall, F1, event-F1, false alarms/day, detection delay, runtime, memory).	Build/insert the metrics page with the final table and comparison plots.	Every number going on a slide must come from this run, not from memory or an earlier test.
28-30	Cache all final outputs locally. Help rehearse; know the model limitations cold.	Prepare screenshots and a backup recorded video in case of live-demo failure. Finalize slides.	Full joint rehearsal of the 2-minute pitch, at least twice.

4. Rules Carried Over From the Draft (These Were Already Right)

- Do not claim ESA-ADB is representative of ISRO by default; do not claim training on private/unreleased ISRO telemetry; state payload housekeeping vs. bus telemetry honestly.
- Every number in the final presentation must come from your team's own run — no borrowed literature numbers presented as your results.
- GaganMitra is human-in-the-loop decision support — it must not issue spacecraft commands or present a model hypothesis as confirmed physical root cause.
- Keep the citation_log.md, data_sources.md, and limitations.md files going from Day 1 — don't reconstruct them the night before submission.
- Freeze scope after the first working end-to-end pipeline (per the draft's own risk mitigation) — resist adding features once Day 12-13 of prep is reached.

Compiled from the team's original execution-plan draft plus the Supernova Grand Finale pitch deck (GaganMitra, Team Mavericks). Next step: confirm Section 1.1 with organizers before starting Day 1.